

Week 6, Lecture 2 WOD: Answer Key

Find the Bugs

NRES 746, Instructor Answer Key (not for student distribution)

Each item repeats the handout's question, then gives the answer, a short rationale, and mistakes to watch for. Both buggy functions are run below, so the symptoms described are real R output. The corrected functions match the Lab 4, Act 2 key, and they are the ones built live in today's demo.

Part 1: Find and fix

1a.

`detect_prob()`: circle the bug, write the corrected line, and say what would go wrong if you ran the buggy version.

Answer. `1 - pobs^k` should be `1 - (1 - pobs)^k`.

Why. An animal is missed on one person-day with probability $1 - p_{obs}$, and missed on all k days with probability $(1 - p_{obs})^k$. The buggy line uses the chance of being *detected* every day instead. The result is a detection probability near 1, so the simulated counts would be close to the true abundance:

```
detect_prob_bug <- function(k, pobs = 0.02) 1 - pobs^k
c(buggy = detect_prob_bug(3), fixed = detect_prob(3))
```

```
##      buggy      fixed
## 0.999992 0.058808
```

Watch for. Writing `(1 - pobs)^k` without the leading `1 -`. That is the probability of being missed, not detected.

1b.

`estimate_power()`: circle the bug, write the corrected line, and say what would go wrong if you ran the buggy version.

Answer. `detected <- ...` should be `detected[i] <- ...`.

Why. Without the index, each pass through the loop overwrites the whole vector with a single `TRUE` or `FALSE`. After the loop, `detected` holds only the last replicate, so the “power” is always exactly 0 or 1:

```
estimate_power_bug <- function(k, s, R = 1000) {
  t_survey <- seq(1, 25, by = s)
  detected <- logical(R)
  for (i in seq_len(R)) {
    counts <- simulate_counts(t_survey, k)
    detected <- detect_decline(t_survey, counts)
  }
  mean(detected)
}
set.seed(3)
replicate(8, estimate_power_bug(3, 1, R = 50))
```

```
## [1] 1 1 0 1 1 1 0 1
```

Watch for. Students who fix it by moving `mean()` inside the loop. That still keeps only one replicate's result.

Part 2: Catch it next time

For one of the bugs, what quick check (one line of code and the answer you'd expect) would have caught it?

Sample answers (either is fine).

- **1a:** `detect_prob(1)` should return 0.02 (one day of effort should equal `pobs`).
- **1b:** run `estimate_power(3, 1)` twice. Each should be a value between 0 and 1 (about 0.8 here), not exactly 0 or 1.

Why. Good checks use cases where you already know the answer: an edge case (`k = 1`) or a range of values the output must fall in. These are the same kinds of checks to demand from AI-written code in today's demo.

Watch for. "Run it and see if the answer looks reasonable." Bug 1a gives counts that look plausible unless you know what to expect. A check needs a specific expected value.